Gpap — **G**PU-**P**arallel **A**ssumption **P**ropagation

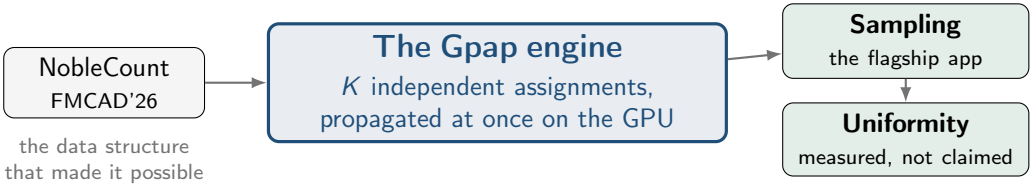
A K-parallel BCP engine — and guaranteed-valid SAT solution sampling built on it

Yogev Shalmon

Technion — Faculty of Data and Decision Sciences

Project status — August 8, 2026

The headline: a BCP engine that runs 300× a CPU core



The result

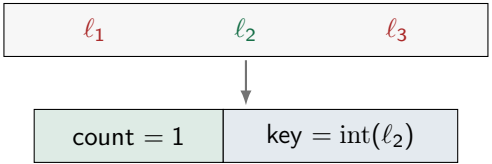
$\sim 10^9$ **propagations/s** vs. $\sim 5 \times 10^6$ on a single CPU core

197–631× — and **essentially flat across every benchmark family**

But an engine alone is not a contribution. So we built the application that needs exactly this shape of parallelism — **SAT solution sampling** — and measured it against the state of the art.

Where this starts: NobleCount (FMCAD'26)

a clause, one literal currently true



two fields per clause, updated branch-free on every flip

count $= 1 \Rightarrow$ the key **is** the surviving literal
$\Rightarrow$ **no clause scan, ever**

Dropped into ProbCCAnr, the SOTA local-search solver. **No heuristic changes, no re-tuning.**

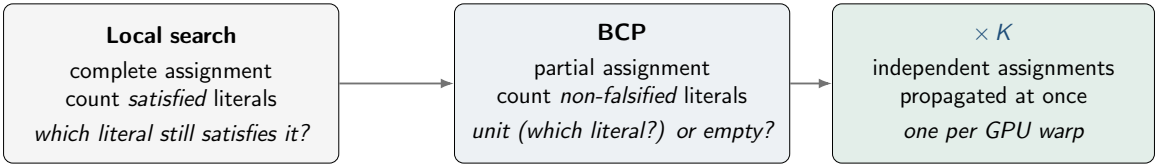
New state of the art on **FCC** (PAR2 -24.8%) *and* **SC22**.

A data-structure win, orthogonal to heuristic design.

Its own closing sentence

"... making it **a promising candidate for parallel clause-state maintenance in SIMD- or GPU-based implementations**."

Same idea, different question



The primitive, in one line

Negated occurrence lists: setting ℓ true falsifies $\neg\ell$ in *exactly* the clauses of $\mathrm{occ}[\ell]$ — a decision touches only what it can affect.

What a GPU is, and what it demands in return

CPU



a **few** clever cores — each fast on its own, each free to do its own thing

GPU



thousands of simple cores — and they move in **lockstep groups of 32**

The deal a GPU offers

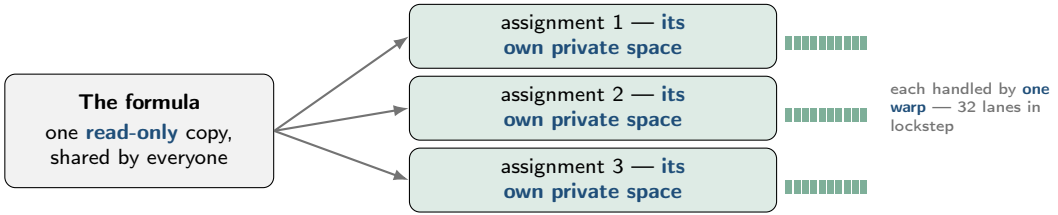
Enormous throughput — **but only if you hand it thousands of independent jobs that all do the same thing.**

And there is a catch. Fetching from memory is slow. A GPU copes by always having *another* job ready to run while one waits.

Run out of jobs and the machine simply idles — however fast its arithmetic is.

So the only question that matters for us is: **how many independent jobs can we supply, and how small can we make each one?**

Our engine in one picture: one job = one assignment



What is in a private space?

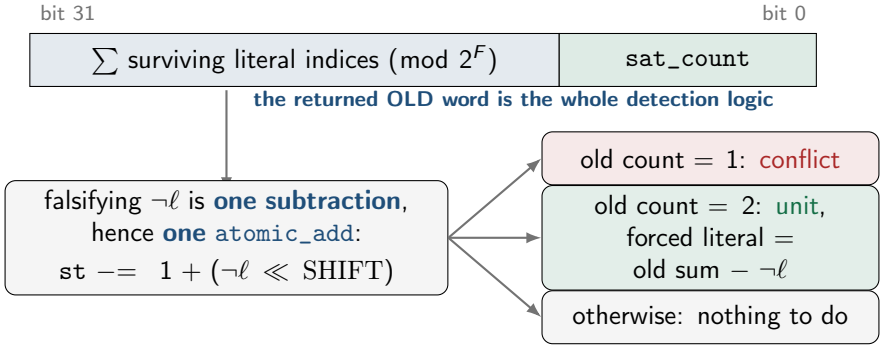
Exactly NobleCount's two numbers per clause — *how many literals are still alive*, and *which one* — packed into **one 32-bit word**. Plus one bit per literal for the assignment itself.

That is the whole trick.

The state is so small (∼4 bytes per clause) that tens of thousands of assignments fit on the card at once — which is precisely the workload the previous slide says a GPU needs.

Private space ⇒ assignments never interfere: no locks, no coordination.

One word per clause — and the update is its own detector



The **assignment** is the same trick: a bitmap where ℓ and $\neg\ell$ share a word, so one `atomic_or` returns duplicate *and* contradiction together.

On a CPU, XOR and sum are equivalent — NobleCount used XOR. On a GPU they are not: the sum makes the update an atomic whose *return value* resolves every race — no locks, no `shfl`/`ballot`.

The architecture: one assignment per warp

Small formula ⇒ ∼100k assignments fit



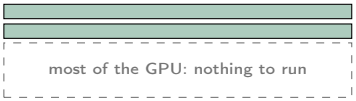
tens of thousands more

The GPU is full — while one assignment waits on memory, thousands of others run. Extra effort per assignment buys nothing.



SCALAR: **one lane of the 32** does the row's work — the other 31 rest

Big formula ⇒ only hundreds fit



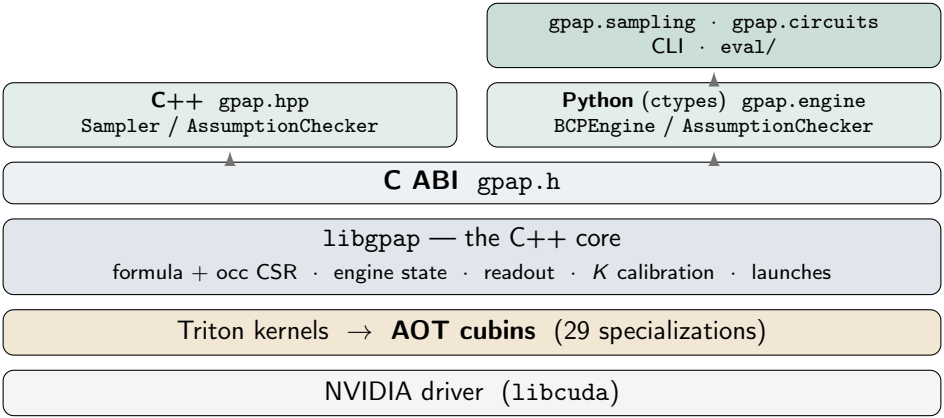
The GPU is starving — too few assignments to keep it busy. **Now we *want* more work per assignment.**



WAVE: **all 32 lanes** pitch in on the *same* assignment

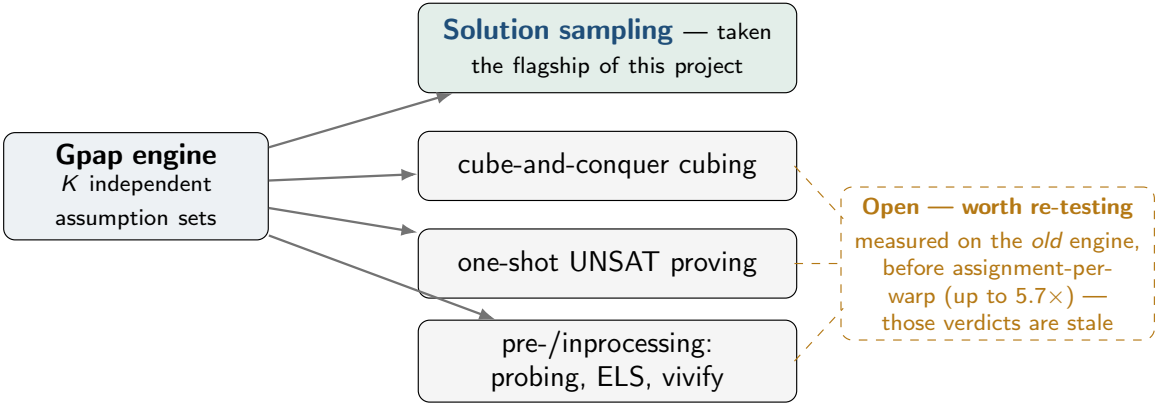
One rule, measured: WAVE when there are too few assignments to fill the GPU, SCALAR once there are plenty. Same data, identical results either way.

The tool



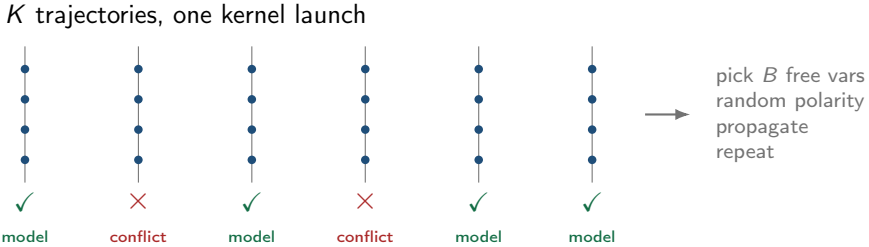
One implementation: both faces link the same library, so there is no second engine to drift. The build needs no CUDA toolkit and no `nvcc`; the runtime needs `libcuda` alone.

What is a K-parallel BCP primitive good for?



Sampling fits the layout exactly: millions of independent randomized completions over one shared, read-only occurrence structure.

The sampler: inspired by CMSGen, run K at a time



Inspired by CMSGen's Algorithm 1 — random variable, random polarity, propagate — but K trajectories at a time.

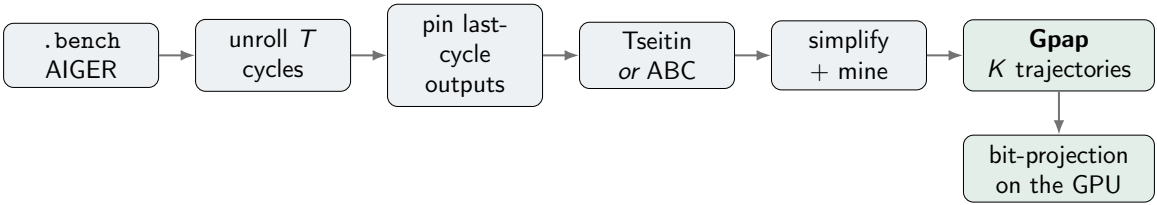
The structural difference: CMSGen backtracks out of a conflict; Gpap *discards* the trajectory.

Two consequences

Every emitted row is a real model — no gradient, no rounding, no repair.

Discarding *is* rejection sampling ⇒ an exactness knob (later).

The flagship pipeline

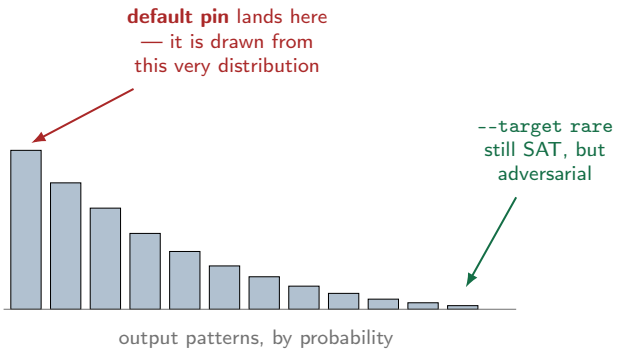


Support = primary inputs × cycles. Only the bit matrix leaves the device.

Encoder is a choice: native Tseitin, or ABC. Same projected solution set — gated by set-equality tests.

Config is per instance: a bandit measures at full K. The right answer is instance-dependent, so we never guess it.

The question we asked before a reviewer could: why not just simulate?



At default pins, plain CPU simulation + filtering beat us on 17/19.

So we changed the protocol, not the claim: pin a *rare* output pattern — still reachable, but adversarial for rejection.

At rare targets it cannot play at all: on EPFL, 0 hits in ∼10M random patterns while Gpap samples at 89–100% success.

Rejection is not just a rival — it picks our targets and audits every emitted sample.

Headline: Gpap vs. CMSGen, 357 cells, 0 errors

family	targets	GPAP / CMSGen	GPAP wins
ITC'99 + ISCAS-89 seq, T=25	**rare**	39.1×	17/18
ISCAS-85 combinational	**rare**	615×	10/10
HWMCC'17, T=25	**rare**	196×	7/7
EPFL combinational	**rare**	9.4×	13/17
Sketch / `sk_` CNFs	—	98.9×	27/29

Rare targets are the honest column.

At easy default pins the ratios are 3–5× higher — so we quote the hard case.

A ratio like this depends on the CPU too.

CMSGen is single-core, so half the number is a statement about the host. We always report the CPU beside it.

Colab L4, 357 cells, 0 skipped, 0 errors, 0 unsound. Per instance at its best encoder. Same suite on a 4080 laptop, and the CPU analysis, in the backup.

Is the speedup the engine, or the benchmark?

	formula size (median)		implications/s		implications/sample	
family (**rare**)	vars N	clauses M	Gpap	CMSGen	Gpap	CMSGen
ISCAS-85 combinational	0.6k	1.4k	1.05×10^9	1.67×10^6	517	507
ITC'99/ISCAS-89, T=25	11k	32k	1.14×10^9	5.20×10^6	30,538	5,422
HWMCC'17, T=25	143k	428k	1.05×10^9	4.58×10^6	12,615	10,737
EPFL combinational	4.5k	13k	1.08×10^9	2.14×10^6	12,139	226
Sketch / `sk_` CNFs	2.6k	11k	1.00×10^9	4.23×10^6	851	355

Largest instances: N up to 413k (sequential) / 391k (HWMCC), M up to 1.15M clauses.

The engine's rate is flat — $\sim10^9$/s on *every* family, from 1.4k to 428k clauses.

So the speedup is the engine, not a lucky benchmark.

Where we sample slower, we are not propagating slower — we are *throwing more away*.

A trajectory that hits a conflict is discarded. On EPFL that wastes far more work than on ISCAS-85, which is exactly why its sampling ratio is the lowest.

Who else puts SAT sampling on a GPU?

DEMOTIC
ASP-DAC'25

ISCAS-85, their own output pins

Recurrent-CircuitSAT
MLCAD'25

closest to our domain: sequential circuits

HTS
arXiv 2502.08673

CNF, hybrid tree search

They all do the same thing: relax the circuit to a *continuous* function and follow the gradient towards a solution.

Fast, and GPU-native — but a gradient lands *near* a solution, so the output must be rounded and repaired, and may not satisfy the formula.

Gpap is discrete throughout: every trajectory is real propagation, so every emitted row is a model by construction — nothing to verify, nothing to repair.

We compare on *their* metric: unique solutions per second of their own reported runtime.

Versus the GPU gradient samplers

rival	pins used	GPAP / rival	GPAP wins
DEMOTIC	*their* ISCAS-85 pins	2.5×	19/20
DEMOTIC	our **rare** pins	2,427×	16/16
Recurrent	*their* target table	4.4×	6/6
Recurrent	our **rare** pins	635×	5/6
HTS	our CNFs	1,584×	13/14

The protocol matters more than the rival

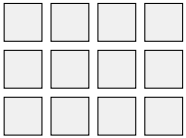
Recurrent-CircuitSAT reads 4.4× on *their* pins and 635× on ours — a 140-fold swing, same sampler, same GPU, same day.

Their pins are easy by construction, so winning there is not a sampling result.

Where we lose: a handful of instances that need clause *learning* — something we deliberately do not do.

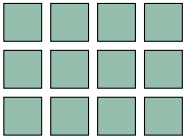
What "uniform" means — and why a sampler lives or dies by it

all solutions of the formula



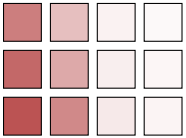
each square = one model

uniform



every model equally likely

skewed



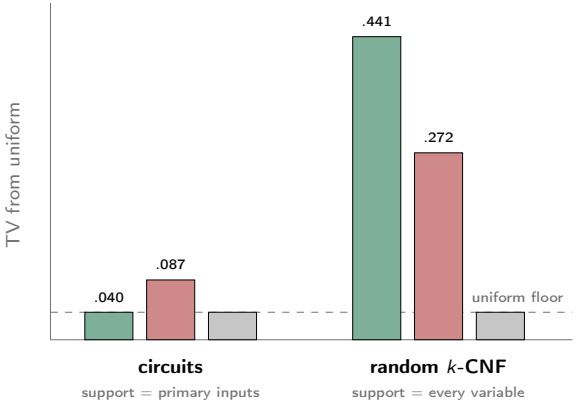
a fast sampler that keeps returning the same corner

Why it matters: sampling is used to *estimate* things — coverage, failure probability, test stimuli. A skewed sampler answers those questions wrongly, no matter how fast it is.

Soundness $\neq$ uniformity. Every row we emit *is* a model — that is free for us. Being drawn with the *right probability* is a separate claim, and it has to be measured.

Standard tool: Barbarik. Ours: an exact oracle (next slide).

Uniformity: measured against an exact oracle



green = Gpap, red = CMSGen, grey = exact-uniform control.

The instrument

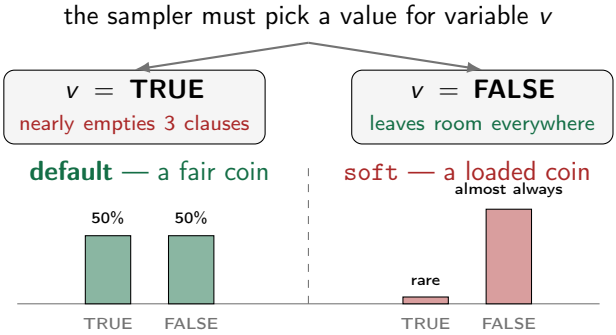
Small formulas whose solutions we enumerate *exhaustively* — so the right answer is known, with no solver in the loop.

On circuits we sit at the floor; CMSGen does not.

On random k-CNF we are worse.

600 instances, 10,800 runs, zero unsound rows.

The `soft` option: trade some uniformity for speed



What `soft` does

Before choosing, it scores both values by how close they push each clause to *empty*, and takes the safer one.

On: far fewer trajectories die ⇒ higher sampling rate.

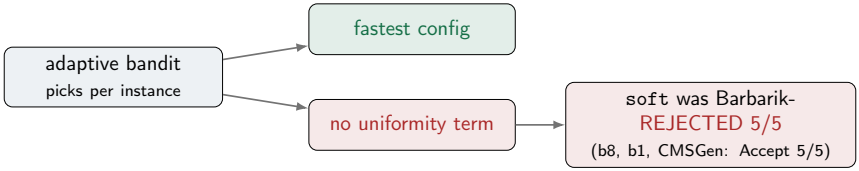
But: the choice is no longer a fair coin, so solutions reachable only through a risky choice are under-sampled.

	TV from uniform	Barbarik
uniform control	0.040	—
Gpap default	0.04–0.22	Accept 5/5
CMSGen	0.18	Accept 5/5
`soft`	0.63–0.76	Reject 5/5

So it is a switch, not a verdict.

Off by default, because a user who asks for a *sampler* should get a uniform one unless they say otherwise. On when throughput matters more — we measured both, and report both.

Why it is off by default: the bandit picks on speed alone



The bandit measures configurations on **models per second** — there is no uniformity term. So with `soft` in the portfolio it gets chosen *silently*, on instances where it happens to be fastest.

Hence the default is off, and turning it on is explicit.

how hard the instance is	cells	cost of staying safe
very hard (succ $< 10^{-4}$)	13	$8\times$ slower
hard	22	$3\times$ slower
moderate	86	$1.2\times$ slower
easy (succ > 0.5)	196	$1.01\times$ — free

On the instances we already handle, the two are within 1% — the difference only appears on the hard ones, which is exactly where `soft` would be chosen.

What `soft` would change in the numbers we publish

family (**rare**)	**default (uniform)**	with `soft`	difference
ITC'99 + ISCAS-89 seq, T=50	14.0×	64.9×	4.6×
ITC'99 + ISCAS-89 seq, T=25	39.1×	74.7×	1.9×
EPFL combinational	9.4×	11.1×	1.2×
HWMCC'17, T=25	196×	203×	1.0×
ISCAS-85 combinational	615×	632×	1.0×
Sketch / `sk_` CNFs	98.9×	102×	1.0×

How we know

The *entire* 357-cell comparison, run twice — one flag changed, same machine, same budgets.

Control: CMSGen ran at 1.004× between the two arms and 0/357 formulas differed, so the effect is `soft` and not the machine.

The difference is real but narrow

Deep sequential unrolls gain 2–5×. Everything else moves by a few percent.

We report the default, and state what the switch buys.

Turning this into a paper

A GPU-native clause-state primitive	one atomic's return value *is* the unit/conflict detection
A batched-assumption BCP engine	K independent assignments, one per warp + a measured dispatch rule
A sampler	guaranteed-valid; 12–538× over CMSGen, 10^2–10^5× over GPU rivals at rare targets
A measurement standard	rejection baseline, target rarity, vacuous pins, emission fairness, uniformity

The evidence is in hand

357 measured cells, 0 errors, 0 unsound samples, three rival head-to-heads, a uniformity campaign against an exact oracle.

Enough to write the experiment section today.

Appendix: every CMSGen family measured on the L4

family	targets	inst	**GPAP/CMSGen**	wins	C=0	G=0	both
ITC'99 + ISCAS-89 seq, T=25	**rare**	19	39.1×	17/18	1	0	1
ITC'99 + ISCAS-89 seq, T=50	**rare**	19	14.0×	15/16	3	0	3
ITC'99 + ISCAS-89 seq, T=25	random	19	183.3×	18/18	1	0	1
ITC'99 + ISCAS-89 seq, T=50	random	19	147.0×	19/19	7	0	0
ISCAS-85 combinational	**rare**	10	615.4×	10/10	1	0	0
ISCAS-85 combinational	random	10	636.1×	10/10	1	0	0
HWMCC'17, T=25	**rare**	7	195.6×‡	7/7	5	0	0
HWMCC'17, T=50	**rare**	7	421.5×‡	7/7	6	0	0
HWMCC'17, T=25	random	7	263.8×‡	7/7	5	0	0
HWMCC'17, T=50	random	7	520.6×‡	7/7	6	0	0
EPFL combinational	**rare**	20	9.4×	13/17	1	1	3
EPFL combinational	random	20	8.2×	12/17	1	1	3
Sketch / `sk_` projected CNFs	—	29	98.9×	27/29	0	1	0

357 cells, 193 instance-slots, 0 skipped, 0 errors. GPU **NVIDIA L4**, CPU **Intel(R) Xeon(R) CPU @ 2.20GHz**. Geomeans already EXCLUDE b02/b05/b07/s526 (outputs constant at T=25, so pinning is a tautology) and any instance that samples with zero search. **The zero columns are symmetric**: **C=0** instances CMSGen could not sample (our wins), G=0 instances *we* could not (our losses), **both** neither could. None has a finite ratio, so none enters a geomean — and the four counts plus the geomean's instances sum to **inst**. ‡geomean over <3 instances (CMSGen scored zero on the rest): an illustration, not an estimate.

Appendix: the engine, isolated from the sampler

			implications/s		implications/sample		**sampling**
family	targets	n	GPAP	CMSGen	GPAP	CMSGen	**GPAP/CMSGen**
ITC'99 + ISCAS-89 seq, T=25	**rare**	13	1.14×10^9	5.20×10^6	30,538	5,422	39.1×
ITC'99 + ISCAS-89 seq, T=50	**rare**	10	1.07×10^9	5.29×10^6	166,856	11,553	14.0×
ITC'99 + ISCAS-89 seq, T=25	random	13	1.10×10^9	4.47×10^6	7,254	5,405	183.3×
ITC'99 + ISCAS-89 seq, T=50	random	9	1.25×10^9	5.38×10^6	14,062	8,867	147.0×
ISCAS-85 combinational	**rare**	9	1.05×10^9	1.67×10^6	517	507	615.4×
ISCAS-85 combinational	random	9	1.05×10^9	1.66×10^6	530	535	636.1×
HWMCC'17, T=25	**rare**	2	1.05×10^9	4.58×10^6	12,615	10,737	195.6×
HWMCC'17, T=50	**rare**	1	0.87×10^9	1.87×10^6	2,953	2,677	421.5×
HWMCC'17, T=25	random	2	1.10×10^9	3.43×10^6	3,522	2,910	263.8×
HWMCC'17, T=50	random	1	0.84×10^9	1.89×10^6	2,253	2,643	520.6×
EPFL combinational	**rare**	12	1.08×10^9	2.14×10^6	12,139	226	9.4×
EPFL combinational	random	13	0.89×10^9	4.53×10^6	15,138	630	8.2×
Sketch / `sk_` projected CNFs	—	27	1.00×10^9	4.23×10^6	851	355	98.9×

GPAP's rate is from the packed-word kernel; CMSGen's from *its own* counters (`-statsevery`). GPAP holds $\sim10^9$ implications/s on every family — spread only 1.41× — while the sample-rate ratio spans $\sim$2 orders of magnitude. The two are linked exactly: **samples/s ratio = (impl/s ratio) / (impl/sample ratio)** — read each row left to right and the last column is what comes out. So the variation is **rejection cost**, not the propagator: we discard a conflicted trajectory, CMSGen backtracks and keeps the partial work.

Appendix: every GPU rival measured on the L4

rival	protocol	cells	GPAP/rival	wins	R=0	G=0	failed
DEMOTIC	their ISCAS-85 pins	20	2.5×	19/20	0	0	0
DEMOTIC	our **rare** pins	18	2,426.6×	16/16	0	0	0
Recurrent	their target table	9	4.4×	6/6	0	0	3
Recurrent	our **rare** pins	9	634.9×	5/6	1	0	3
HTS	our CNFs (both encoders)	18	1,584.4×	13/14	2	1	3

Their metric: unique solutions / their `model_runtime`, which excludes their model build while ours includes construction. **failed** = timeout or no gradient path; those rows are reported, never folded in as a zero. **R=0** the rival emitted nothing, G=0 *we* emitted nothing — neither has a finite ratio, both are counted.

Appendix: uniformity against an exact oracle

sampler	circuit		random k-CNF	
	median TV	skewed	median TV	skewed
`ctl:uniform`	0.0395	0/36	0.0401	0/36
`ctl:tv0.05`	0.0577	36/36	0.0580	36/36
`gpap:b1`	0.1109	18/36	0.4368	36/36
`gpap:b8`	0.0401	10/36	0.4033	36/36
`gpap:soft`	0.6255	24/36	0.7605	36/36
`gpap:b32`	0.0399	0/36	0.0401	0/36
`gpap:b64`	0.0399	0/36	0.0401	0/36
`cmsgen`	0.0945	25/36	0.2576	36/36

72 generated instances whose ENTIRE solution set is enumerated by brute force, so the target is known exactly; χ^2 over 3 seeds, Fisher-combined, $\alpha = 0.001$. 0 unsound emissions. **The controls calibrate the instrument**: `ctl:uniform` is exactly uniform and must NOT be flagged; `ctl:tv0.05` is skewed by a known amount and must be. `soft` is why it is opt-in — and $B \geq |\text{support}|$ (b32/b64) is exactly uniform, because discarding conflicts IS rejection sampling.